Module 03 - Report

Stefano Vitale - ID: 654693

1 Workload Characterization and Data Skew

To satisfy the requirement of a non-uniform workload distribution across partitions, it was necessary to inject frequency skew into the dataset through massive key duplication. Due to the deterministic nature of any hash function, identical keys are inevitably mapped to the same partition.

Consequently, this duplication renders the uniform distribution properties of `murmur3 fmix32` irrelevant. However, this approach is strictly necessary to force a structural bottleneck and accurately evaluate the system's resilience and the load-balancing effectiveness of the OpenMP Task paradigm.

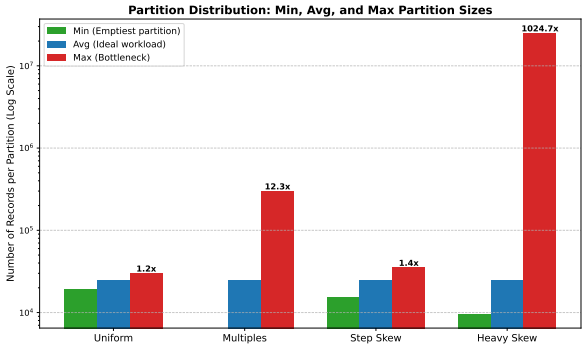
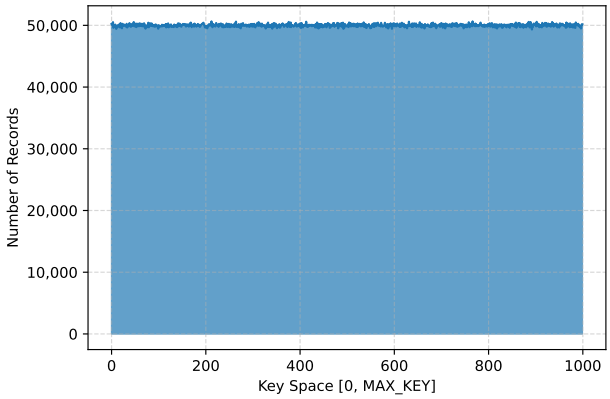


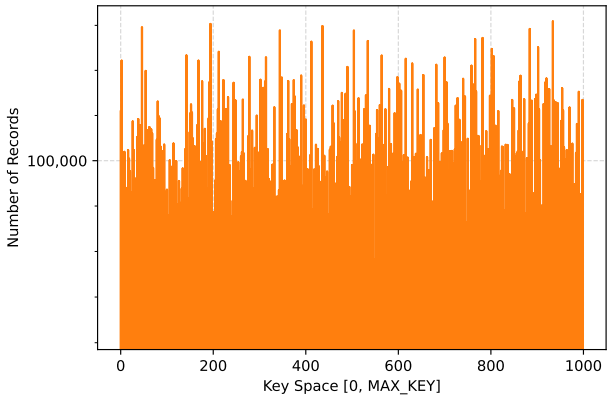
Figure 1: Partition load imbalance **after** Murmur3 hashing (Log Scale).

Dataset Distributions

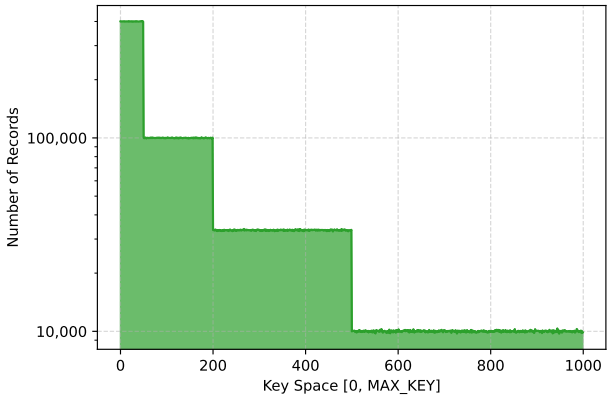
As shown in Figure 2, four deterministic generation models are implemented:



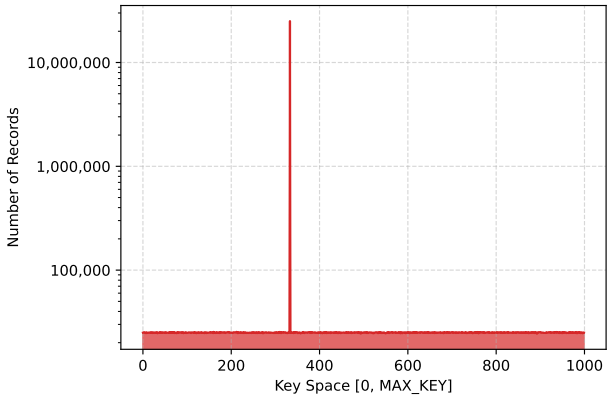
(a) Uniform: Ideal baseline, keys distributed evenly.



(b) Multiples: Keys are restricted to exact multiples of 1,000.



(c) Step Skew: The key space is divided into tiers to create gradual imbalance.



(d) Heavy Skew: 50% of the entire dataset is forced into a single, heavily duplicated key.

Figure 2: Raw input data distribution before hashing (aggregated into 1000 visual bins).

2 OMP Implementation Strategies

This section describes how the algorithmic choices and manual thread management strategies defined in the parallel baseline version (`hashjoin_par.cpp`) were translated into OpenMP constructs.

OpenMP Loop-Based (`hashjoin_omp_loop.cpp`) This implementation maps the baseline's logic into OpenMP loop constructs. For the **Histogram and Scatter phases**, the manual calculation of static array chunks is directly replaced by `#pragma omp parallel for schedule(static)`, followed by a sequential global reduction. For the **Join phase**, the manual atomic counter is replaced by the native runtime scheduler via **schedule(dynamic, 1)**. This assigns the P partitions dynamically, one at a time, to the T available threads, providing fine-grained load balancing with minimal code overhead.

OpenMP Task-Based (`hashjoin_omp_task.cpp`) This implementation replaces manual thread management with OpenMP task constructs. For the **Histogram and Scatter phases**, the static chunking paradigm is preserved by enforcing a 1:1 mapping: exactly T explicit tasks are generated to process the static data blocks, and with exactly T tasks and T threads available, the runtime is expected to assign one task per thread; while OpenMP provides no formal guarantee, this holds in practice under low contention and is consistent with the observed performance. For the **Join phase**, the dynamic distribution is achieved by spawning P independent tasks within a `single` construct. To translate the global aggregation safely, the manual reduction is replaced by the OpenMP syntax `#pragma omp taskgroup task_reduction` coupled with the `in_reduction` clause on individual tasks.

Runtime Environment Configuration In all execution scripts, `OMP_NUM_THREADS` is mapped to the `-t` parameter, with `OMP_PROC_BIND=spread` and `OMP_PLACES=cores` to distribute threads evenly across physical cores and maximize memory bandwidth utilization.

3 Performance Evaluation

To evaluate the baseline performance of the three parallel architectures, an **input scaling test** was conducted using a **uniform distribution**. The architectural parameters were strictly fixed to $P = 2048$ partitions and $T = 16$ threads across all tests to observe how runtime overhead affects overall efficiency as the computational workload increases. To ensure a rigorous evaluation of the hardware overhead, the key domain (max-key) is scaled proportionally to the dataset size to maintain a constant expected multiplicity factor of 100 across all tests. This isolates the runtime overhead from any algorithmic variations in the Probe phase.

Size (N)	Version	Time (ms)	Speedup	Eff
500.000	Sequential	22.0	-	-
	Threads	9.4	2.34x	14.6%
	OMP Loop	6.9	3.20x	20.0%
	OMP Task	20.1	1.10x	6.9%
5.000.000	Sequential	295.4	-	-
	Threads	34.5	8.56x	53.5%
	OMP Loop	31.0	9.54x	59.6%
	OMP Task	31.4	9.42x	58.9%
50.000.000	Sequential	3168.2	-	-
	Threads	270.8	11.70x	73.1%
	OMP Loop	262.4	12.07x	75.5%
	OMP Task	266.0	11.91x	74.4%

Table 1: Performance comparison across different input sizes with $P = 2048$ and $T = 16$.

For the small dataset, parallel efficiency is low. The fixed costs of parallelization shadow the benefits, especially in `OMP Task` where scheduling 2048 tasks results in a speedup of only 1.10×. Tuning P or T for such small inputs would not yield substantial improvements due to these unavoidable fixed startup penalties.

For larger workloads ($N = 50M$), overheads become negligible. All implementations converge to an optimal speedup of $\sim 12\times$ (75% efficiency), proving that task-based management is not a bottleneck at scale.

Execution Phase Breakdown

To understand the internal bottlenecks of the algorithms, a **phase-by-phase breakdown** was extracted from the Uniform distribution ($N = 50M$, $P = 2048$, $T = 16$). As illustrated in Figure 3, the `prefix_sum` phase is entirely negligible (< 0.05 ms). The `scatter_part` phase dominates the execution time across all implementations (~ 170 ms), primarily due to the massive volume of memory writes and the resulting saturation of the memory bus.
More importantly, the partitioning phases (`histogram` and `scatter_part`) execute a purely linear scan of the input arrays. Their computational complexity is strictly $O(N)$ and is entirely independent of the used distribution of the keys. Therefore, **any performance degradation caused by data skew will be isolated within the** `join_partition` phase, where hash collisions and local bucket sizes dictate the workload.

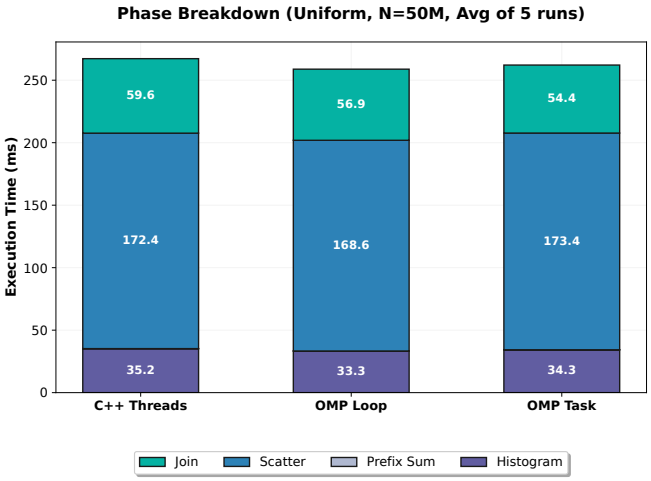


Figure 3: Execution time breakdown of the Hash Join phases (Uniform Distribution).

To evaluate the robustness of the dynamic load-balancing strategies, the algorithms were tested against four increasingly skewed distributions. Given the linear nature of the partitioning phases, this analysis isolates the `join_partition` time (Figure 4).

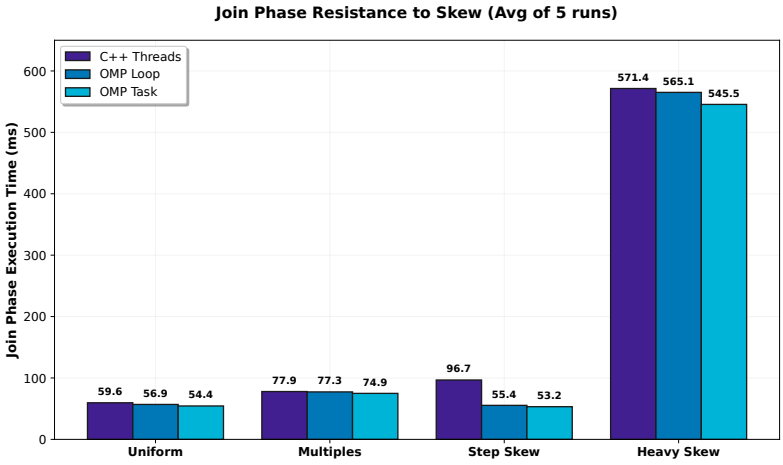


Figure 4: Performance degradation of the Join phase across different data distributions.

Moderate imbalances (*Multiples* and *Step Skew*) are effectively handled by dynamic scheduling and task queues, keeping the Join phase under 100 ms.

However, **Heavy Skew** exposes a limit in parallel granularity: most data falls into a single partition. Since 1 task equals 1 partition, the thread assigned to it becomes a sequential bottleneck. While 15 threads exhaust their partition queue and remain idle, one thread processes the massive bucket alone.

This behavior perfectly illustrates Amdahl's Law: the parallel efficiency is strictly bounded by the largest indivisible sequential fraction of the workload. The execution time of the Join phase spikes from ~ 55 ms to over 540 ms across all implementations, proving that while OpenMP provides excellent low-overhead scheduling, it cannot overcome algorithmic bottlenecks rooted in coarse-grained work items. To survive Heavy Skew, a different approach would be required, such as recursively partitioning the overloaded buckets (Nested Tasks) to re-enable parallel work-stealing (illustrated in section 6 as an experiment).

4 Correctness and Methodology

To ensure a rigorous and fair evaluation, both OpenMP implementations were strictly validated against the optimized sequential baseline and the manual `C++ Threads` version developed in Module 2. Tests via a dedicated verification script (`naive_check.sh`) confirmed that all implementations yield identical outputs (`join_count`, `checksum1`, and `checksum2`) across all uniform and skewed datasets.

To eliminate OS-level noise, all metrics represent the arithmetic **mean of 5 consecutive executions**.

To guarantee a fair comparison, the `OMP Loop` and `OMP Task` versions were designed to enforce a strict 1:1 structural equivalence with the manual `std::thread` baseline. By freezing the algorithmic logic (e.g., maintaining bucket-level task granularity), the preceding analyses isolated the real overhead of the concurrent execution models. The parallel baseline was launched without explicit affinity pinning, matching its default behavior from Module 2.

5 Scalability Analysis

To rigorously evaluate the performance bounds of the OpenMP constructs compared to the baseline `C++ Threads` implementation, both Strong and Weak scaling analyses were conducted using the **Uniform distribution**. All reported metrics represent the arithmetic mean of 5 consecutive executions to eliminate OS-level noise.

5.1 Strong Scaling

In the strong scaling evaluation (Figure 5), the problem size was strictly fixed at $N = 50M$ tuples. The analysis was extended up to $T = 40$ threads:

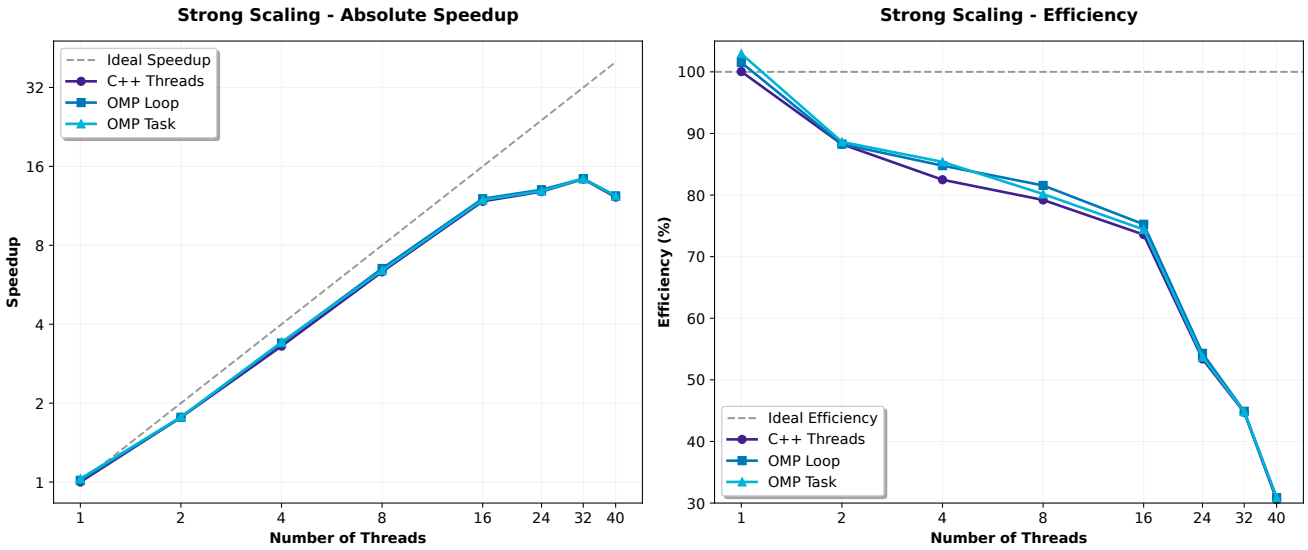


Figure 5: Strong Scaling evaluation. Left: Absolute Speedup against the ideal linear boundary. Right: Absolute Efficiency percentage.

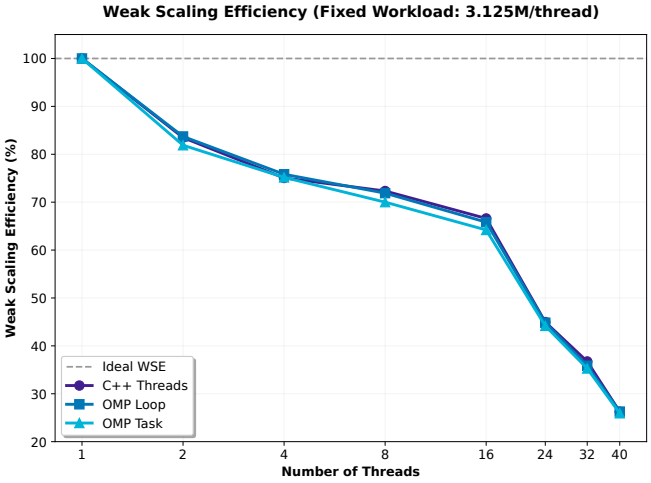
The scaling curve perfectly reflects this hardware topology:

- **Phase ($T \leq 16$):** Mapping one thread per physical core yields the best efficiency in this range, achieving a $\sim 12\times$ speedup with $\sim 75\%$ efficiency.
- **Phase ($16 < T \leq 32$):** Minimum execution time ($\sim 0.220\,\text{s}$) at 32 threads. HT does not increase memory bandwidth but hides cache-miss latency, yielding a modest $\sim 15\%$ improvement over the 16-core configuration.
- **Phase ($T > 32$):** At 40 threads, the OS is forced into continuous context switching. This overhead degrades overall execution time, as the OS scheduler introduces latency and increases memory access contention among oversubscribed threads.

Crucially, the curves of all three implementations overlap almost perfectly. Because the Hash Join is a memory-bound algorithm, the primary physical bottleneck is the memory bus bandwidth, not the software scheduler.

5.2 Weak Scaling

The weak scaling benchmark (figure below) evaluates the system's ability to handle proportional workload increases.



Beyond the physical core count ($T > 16$), adding threads scales the dataset but not the memory bandwidth.

From $T = 1$ to 16, the WSE stabilizes above 60%. However, in the **oversubscription domain** ($T > 32$), efficiency collapses to $\sim 25\%$. Pushing $125M$ records through a saturated bus increases cache evictions and contention.

Despite this, `OMP Task` and `OMP Loop` mirror the `C++Threads` baseline.

6 Alternative for Heavy Skew Mitigation: Nested Task Version

Up to this point, all OpenMP implementations were strictly constrained to a 1:1 algorithmic mapping with the baseline C++ version to ensure an extremely fair comparison. However, the Heavy Skew scenario exposed the physical limits of bucket-level granularity. In this section, **the 1:1 mapping constraint is broken** to introduce a dynamic skew detection mechanism (`OMP Nested`).

When a thread dequeues a partition, it compares its size against a fixed threshold of 3× the expected uniform bucket size ($3 \times \frac{N}{P}$). If the partition exceeds this limit, the thread enters Nested Mode. The Hash Table Build phase remains strictly sequential. Crucially, once built, the `SimpleHashTable` is completely read-only, making it natively thread-safe for concurrent lock-free lookups. The Probe array is then divided into micro-chunks ($4 \times T$), and the thread spawns nested `#pragma omp task`s for each chunk, waking up the idle threads and allowing them to pick up the newly spawned chunks.

Phase	**OMP Loop (ms)**	**OMP Task (ms)**	**OMP Nested (ms)**
Histogram	32.83	33.91	33.35
Prefix Sum	0.03	0.03	0.02
Scatter Part	122.15	125.74	125.59
Join Partition	**563.91**	**543.69**	**377.40**
Total Time (ms)	722	706	**539**
Speedup	3.91x	3.99x	**5.23x**

Table 2: Phase breakdown under Heavy Skew. Nested Tasks reduce the Join bottleneck by $\sim 33\%$.

7 Conclusion: Loop vs. Task

Under a strict 1:1 partition-to-thread mapping for memory-bound workloads, the explicit task model provides no performance advantage over loop scheduling, and can become actively harmful when task granularity is too fine.

The root cause is structural: the OMP Task runtime incurs an initial overhead that grows linearly with P, allocating P tasks and managing the `taskgroup` reduction barrier, while useful work per task scales as $O(N/P)$. When N is small or P is large, overhead dominates. Table 3 quantifies this: at N=500K and P=4096, OMP Task is 4.17× slower than OMP Loop and falls *below the sequential baseline* (speedup 0.77×). As N grows, the two schedulers converge: at N=5M the ratio drops to ≈1.00× for P=2048, and at N=50M all configurations converge to within 1–2% regardless of P.

Table 3: Ratio > 1: Loop faster; ≈1: convergence. †: Task slower than sequential.

N	P	Loop (ms)	Task (ms)	Ratio
500K	64	6.3	6.9	1.09×
	2048	7.1	20.0	**2.82**×
	4096	8.6	36.1	**4.17**×†
5M	64	46.5	50.6	1.09×
	2048	31.6	31.7	≈1.00×
	4096	35.3	36.3	1.03×
50M	64	584	646	1.11×
	2048	288	291	≈1.01×
	4096	298	302	≈1.01×